

Data Ingestion

Code Reference - complete flow for DOCX, Excel and Publuu

MASTER FLOW - FILE SOURCES (DOCX / EXCEL)

Admin -> `upload_file()` -> **Supabase Storage** -> `run_storage_sync_loop()` -> `synchronize_storage()` -> `downloaded_source_files()` -> **change detection** -> `_parse_source_file()` -> **DOCX or Excel processing** -> **validation** -> `insert_new_activities()` -> `register_processed_source()`.

MASTER FLOW - PUBLUU

Admin URL -> `add_external_source()` -> `external_sources` -> **Admin clicks Test Source** -> `test_external_source()` -> `extract_activities_from_publuu()` -> **PDF bytes** -> **page images** -> **Vision extraction** -> **Preview** -> **Admin review/edit** -> `save_external_source_preview()` -> **validation** -> `insert_new_activities()` -> `activities`.

THREE DATABASE TABLES TO REMEMBER

- `activities` - final unified activity records from every source.
- `ingested_sources` - processing metadata for DOCX/Excel: file name, hash, source type, activity count, status.
- `external_sources` - external source metadata: Publuu name, URL, active flag, last sync/save status.

CURRENT RUNTIME BEHAVIOR

- DOCX/Excel sync runs in the background with the Telegram application and checks again about 60 seconds after the previous cycle finishes.
- Publuu is not processed by that storage watcher. In the current implementation, extraction starts when the admin clicks Test Source.
- After a file-source sync, `refresh_data=reload_data` refreshes the activity data used by the search layer.

END RESULT

Different source formats use different readers and parsers, but all successful paths end as the same unified Activity schema in the `activities` table. The raw source files are only ingestion inputs; the Agent later searches normalized activities.

1. File Upload, Storage and Synchronization

1.1 Admin upload

- UI file: `admin/templates/index.html`.
- The upload form sends HTTP POST to `upload_file()` in `admin/app.py`.
- The Flask backend reads the uploaded file as bytes and stores the original file in Supabase Storage.
- No DOCX/Excel parsing happens inside `upload_file()`.

```
uploaded_file = request.files.get("file")
file_bytes = uploaded_file.read()
_get_storage_bucket().upload(file_name, file_bytes, {...})
```

1.2 Where the background sync starts

- `agent/telegram_bot.py` starts the sync task in `_start_storage_sync()`.
- It calls `run_storage_sync_loop(refresh_data=reload_data)`.
- `storage_watcher.py` controls when a check happens; `storage_sync.py` controls what changed and what action is required.

```
run_storage_sync_loop(...)
synchronize_storage_once()
synchronize_storage()
```

1.3 Reading the old state and current state

- Old processed-source state: `get_processed_sources()` reads table `ingested_sources`.
- Current Storage state: `downloaded_source_files()` downloads current source files from Supabase Storage.
- Each downloaded file is returned as bytes, written to a temporary local file, and then passed through the rest of the pipeline as a `Path`.

```
file_bytes = client.storage.from_(bucket_name).download(file_name)
local_path.write_bytes(file_bytes)
yield downloaded_files
```

1.4 Change detection

- File names are compared first.
- For names that exist in both places, the current SHA-256 is compared with the stored `file_hash`.
- The result is grouped into added, replaced, deleted, and unchanged.
- Hash logic is in `ingestion/file_hash.py`.

1.5 Apply the change safely

- `_apply_added_file()` parses a new file, saves activities, then registers source metadata.
- `_apply_replaced_file()` parses the new file before deleting the old data. If saving fails, the previous activities and metadata are restored.
- `_apply_deleted_file()` removes activities and source metadata; if the operation fails, previous data is restored.

2. DOCX Processing Path

2.1 Branch to DOCX

File: ingestion/storage_sync.py

```
def _parse_source_file(file_path):  
    if suffix in SUPPORTED_DOCX_SUFFIXES:  
        activities = parse_ai_docx(file_path)
```

2.2 DOCX controller

- File: ingestion/ai_docx_parser.py.
- Function: parse_ai_docx(file_path).
- Checks that the path exists and that the suffix is .docx.
- Calls parse_generic_schedule(file_path) and later runs the final DOCX validation.

2.3 Where the Word file is actually opened

- parse_generic_schedule() -> _build_document_content(file_path) -> read_docx(file_path).
- File: ingestion/readers/docx_reader.py.
- The real file-open operation is Document(file_path).
- The reader extracts paragraphs and tables and returns a Python structure.

```
document = Document(file_path)  
return {"source_file": file_path.name, "paragraphs": paragraphs, "tables": tables}
```

2.4 What is sent to the LLM

- The Word file itself is not sent to the model.
- The extracted paragraphs and tables are placed in a Python dictionary and serialized with json.dumps().
- That JSON text is inserted into the extraction prompt.
- The model is gpt-4o-mini with temperature 0 and structured output.

```
serialized = json.dumps(content, ensure_ascii=False, indent=2)  
extraction = EXTRACTOR.invoke(prompt)
```

2.5 Structured output and deterministic cleanup

- The LLM returns GenericDocumentExtraction with center_name, source_language, and a list of GenericActivityCandidate.
- _normalize_candidate() normalizes text, day, time, status, branch/location and required fields.
- Candidates missing essential fields such as name, day, start time, or center name are rejected.
- _deduplicate() removes duplicate activities.

2.6 Final DOCX validation

- `ai_docx_parser.py` calls `validate_activities(activities)`.
- If the validation report does not pass, `parse_ai_docx( )` returns an empty list.
- Therefore the DOCX path is strict: an invalid final extraction does not continue to database saving.

3. Excel Processing Path

3.1 Branch to Excel

File: ingestion/storage_sync.py

```
elif suffix in SUPPORTED_EXCEL_SUFFIXES:  
    activities = ingest_structured_file(file_path)
```

3.2 Structured ingestion controller

- File: ingestion/source_ingestion.py.
- Function: ingest_structured_file(file_path).
- It validates the local temporary path and Excel suffix, then calls read_excel_activities(path).

3.3 Where Excel is actually opened

- File: ingestion/readers/excel_reader.py.
- Pandas reads the workbook with openpyxl.
- If a sheet named activities exists it is preferred; otherwise the first sheet is used unless a specific sheet was requested.
- The selected DataFrame is converted into a list of row dictionaries.

```
workbook = pd.read_excel(path, sheet_name=None, engine="openpyxl")  
records = dataframe.to_dict(orient="records")
```

3.4 Map each row to the unified schema

- Each record is passed to adapt_activity_record() in ingestion/source_adapter.py.
- FIELD_ALIASES maps different source column names to the same internal field.
- Example: activity_name, title, and other known aliases can map to the unified name field.
- The adapter performs deterministic normalization for day, time, text, age, status and language.

3.5 End-time inference is deterministic

- If an explicit end time is present, it is used.
- If a duration is available, infer_end_time() may calculate the end time according to deterministic source rules.
- No LLM is used in the Excel path.

3.6 Excel validation behavior

- `validate_activities()` creates a validation report.
- `keep_valid_activities()` keeps only records without critical errors.
- `_deduplicate_activities()` then removes duplicates.
- Important difference: Excel can continue with a valid subset even if some rows are invalid.

4. Publuu Processing Path

4.1 Add the external source

- UI file: `admin/templates/index.html`.
- The admin enters a source name and Publuu URL.
- The form calls `add_external_source()` in `admin/app.py`.
- `create_external_source()` stores the metadata in the `external_sources` table.
- At this stage no PDF is downloaded and no AI extraction runs.

4.2 Start extraction

- The admin clicks Test Source.
- `test_external_source(source_id)` loads the source by ID and calls `extract_activities_from_publuu(source["url"])`.
- This is the current trigger for Publuu extraction.

4.3 Find and download the real PDF

- File: `ingestion/readers/publuu_reader.py`.
- `download_publuu_pdf()` loads the public Publuu page with HTTP GET.
- The HTML is scanned for candidate links from attributes such as `href/src/data-url/data-download` and strings containing `.pdf` or `download`.
- Candidate URLs are tried until the response is confirmed as PDF by content signature or content type.
- The result is returned as PDF bytes.

```
if body.startswith(b"%PDF") or "application/pdf" in content_type:
    return body
```

4.4 PDF bytes remain in memory

- Unlike DOCX/Excel, the Publuu PDF is not written to a temporary local file and is not stored in Supabase Storage.
- `pymupdf.open(stream=pdf_bytes, filetype="pdf")` opens the PDF directly from memory.

4.5 Render each page into images

- `_render_full_page()` creates one full-page PNG image.
- `_render_page_sections()` creates three enlarged horizontal sections with a small overlap.
- `SECTION_COUNT = 3`.
- The page is visually rendered and cropped; the PDF bytes themselves are not split into three pieces.

5. Vision Extraction, Preview and Save

5.1 Prepare images for the Vision request

- The page images exist as PNG bytes in Python memory.
- `_image_part()` Base64-encodes the PNG bytes and builds an `input_image` data URL.
- For each section, the model receives two images: the full page for context and one enlarged section for precise reading.

```
encoded = base64.b64encode(image_bytes).decode("ascii")  
"image_url": "data:image/png;base64," + encoded
```

5.2 Actual Vision model call

- File: `ingestion/readers/publuu_activity_extractor.py`.
- Function: `_extract_section()`.
- The actual API call is `client.responses.create(...)`.
- The request includes text instructions, full-page image, section image, and a strict JSON schema named `ACTIVITIES_SCHEMA`.

```
response = client.responses.create(  
    model=DEFAULT_MODEL,  
    input=[...full_page..., ...section_image...],  
    text={"format": {"type": "json_schema", ...}}  
)
```

5.3 Merge and clean the extracted results

- `_parse_activities()` parses the structured JSON output.
- Section results are appended into `page_activities`.
- `_fill_page_shared_fields()` fills shared page context when safe.
- `_deduplicate_activities()` merges duplicates caused by overlapping sections.
- `_apply_deterministic_cleanup()` performs safe cleanup.
- After all pages, `_normalize_document_consistency()` normalizes consistent document-wide values.

5.4 Human review before database save

- `test_external_source()` renders `external_preview.html`.
- At this point the extracted activities are only shown to the admin; they are not yet saved to `activities`.
- The admin can review and edit the extracted values.

5.5 Save the reviewed Publuu activities

- `save_external_source_preview()` reads the edited form values with `_read_edited_external_activities()`.
- `_adapt_external_activities()` converts them to the unified Activity schema.
- `validate_activities()` must pass before saving.
- Then `insert_new_activities()` writes the new activities and `update_external_source_sync_status()` marks the external source as saved.
- If replacing previous activities fails, the old activities are restored.

6. Saving, Metadata and Data Representation

6.1 Final activity save

- File: `database/activities_repository.py`.
- Function: `insert_new_activities(activities)`.
- It checks required fields, prepares only columns that exist in the table, and prevents duplicates against both the database and the current batch.
- Final insert: `client.table("activities").insert(new_records).execute()`.

6.2 Register processed DOCX/Excel source metadata

- File: `database/ingested_sources_repository.py`.
- Function: `register_processed_source()`.
- Stores `source_file`, `file_hash`, `source_type`, `activities_count`, and `status="processed"`.
- This metadata is what the next sync cycle uses as the previous processed state.

6.3 Representation cheat sheet

- Admin upload: browser file -> Flask uploaded file -> bytes.
- Supabase Storage download for DOCX/Excel: bytes -> temporary local file -> Path.
- DOCX reader: temporary DOCX path -> paragraphs/tables -> Python dict -> JSON text -> LLM prompt.
- Excel reader: temporary Excel path -> Pandas workbook/DataFrame -> row dictionaries -> unified activities.
- Publuu: URL -> HTML -> PDF bytes in memory -> PyMuPDF pages -> PNG bytes -> Base64 -> Vision API.
- Final data for every source: Python activity dictionaries -> Supabase PostgreSQL activities table.

6.4 Storage vs Database

- Supabase Storage contains the original DOCX/Excel source files.
- Supabase PostgreSQL contains normalized activity records and source metadata.
- Publuu PDF is not permanently stored in Supabase Storage in the current path.

6.5 What happens after ingestion

- The Agent does not reopen DOCX, Excel or Publuu PDF files for each user question.
- The ingestion pipeline prepares normalized activities first; the question-processing path later works with the activity data.

7. Show Me in the Code

נושא	File	Function / Line
העלאת קובצי וורד וגיליון	admin/app.py	upload_file()
חיבור למאגר קובצי המקור	admin/app.py	_get_storage_bucket()
לולאת הסנכרון ברקע	ingestion/storage_watcher.py	run_storage_sync_loop()
לוגיקת הסנכרון הראשית	ingestion/storage_sync.py	synchronize_storage()
הורדה זמנית של קובצי המקור	ingestion/storage_source.py	downloaded_source_files()
קובץ חדש מוחלף או שנמחק זיהוי	ingestion/storage_sync.py	_detect_changes_from_maps()
בחירת מסלול לפי סוג הקובץ	ingestion/storage_sync.py	_parse_source_file()
ניהול מסלול מסמך וורד	ingestion/ai_docx_parser.py	parse_ai_docx()
פתיחת קובץ הוורד	ingestion/readers/docx_reader.py	read_docx() / Document(file_path)
הקריאה בפועל למודל השפה	ingestion/generic_llm_parser.py	EXTRACTOR.invoke(prompt)
פתיחת קובץ הגיליון	ingestion/readers/excel_reader.py	pd.read_excel()
מיפוי עמודות הגיליון	ingestion/source_adapter.py	FIELD_ALIASES / adapt_activity_record()
הוספת כתובת המקור החיצוני	admin/app.py	add_external_source()
התחלת חילוץ מהמקור החיצוני	admin/app.py	test_external_source()
איתור והורדת המסמך	ingestion/readers/publuu_reader.py	download_publuu_pdf()
פתיחת המסמך מתוך הזיכרון	ingestion/readers/publuu_activity_extractor.py	pymupdf.open(stream=pdf_bytes, ...)
הקריאה בפועל למודל הראייה	ingestion/readers/publuu_activity_extractor.py	_extract_section() / client.responses.create()
תצוגה מקדימה לפני שמירה	admin/app.py	test_external_source() -> external_preview.html
נתוני המקור לאחר בדיקה שמירת	admin/app.py	save_external_source_preview()
הפעילויות במסד הנתונים שמירת	database/activities_repository.py	insert_new_activities()
רישום פרטי מקור הקובץ	database/ingested_sources_repository.py	register_processed_source()

ONE SENTENCE TO REMEMBER

The ingestion architecture separates source-specific reading from the unified data model: DOCX uses semantic extraction, Excel uses deterministic structured mapping, and Publuu uses Vision extraction plus human preview, but every successful path ends in the same activities table.